

***Построение самовосстанавливающегося
отказоустойчивого кластера PostgreSQL на
базе Patroni/etcd в Yandex Cloud с мониторингом,
автоматическим восстановлением и
оповещением об инцидентах***

OTUS-PostgresAdvanced-2026-04



**Меня хорошо видно
& слышно?**



Тема: Построение самовосстанавливающегося отказоустойчивого кластера PostgreSQL на базе Patroni/etcd в Yandex Cloud с мониторингом, автоматическим восстановлением и оповещением об инцидентах



Брухан Дарья



DevOps/SRE Engineer с 8 годами опыта в IT и сильной практической экспертизой в облачной инфраструктуре, системном администрировании и базах данных. Специализируюсь на работе с Azure и AWS, CI/CD, диагностикой production-систем и СУБД (Oracle, MySQL, MS SQL, PostgreSQL).
Обладательница сертификатов AZ-900, AZ-104

План защиты

Цель и задачи проекта

Какие технологии
использовались

Что получилось

Выводы

Вопросы и рекомендации

Цель и задачи проекта

Цель проекта: Построить отказоустойчивый кластера PostgreSQL на базе Patroni/etcd в Yandex Cloud используя Terraform + Ansible с мониторингом Prometheus Stack

1. Развернуть инфраструктуру в Yandex Cloud при помощи Terraform.
2. Автоматизировать разворачивания PostgreSQL кластера Patroni используя унифицированные роли Ansible
3. Настроить систему мониторинга Prometheus Stack с оповещением в Telegramm, по средствам Alertmanager. Визуализировать данные в виде дашборда в Grafana
4. Провести отладочные работы используя разные варианты падения кластера или отказа одного из узлов.
5. Сделать выводы по результатам проделанных шагов и провести работу на ошибками.

Использовались следующие технологии:

1. Yandex Cloud
2. Terraform
3. Ansible
4. Python 3.14
5. PostgreSQL (Patroni, HAProxy, etcd)
6. Prometheus + Alertmanager + Grafana



Шаги реализации проекта

Этап 1

Настроила консольную утилиту Terraform на локальном ПК. После чего описала будущие виртуальные машины в Terraform используя провайдер Yandex Cloud.

Этап 2

Произвела поиск готовых ролей для установки postgresql и prometheus стека используя ansible, и оптимизировала их под свою инфраструктуру с фиксацией ошибок совместимостей сущностей postgresql+patroni.

Этап 3

Выполнила настройку prometheus стэка с настройкой оповещения через Alertmanager в Telegram.
Импортировала дашборды Grafana для отображения статуса кластера postgresql(patroni/etcd/haproxy)

Этап 4

Протестировала падения кластера и отработку автоматизированного восстановления с помощью сервиса написанного на python и оповещения.

Этап 5

Зафиксировала ошибки и провела часть работы по исправлению ошибок в дипломе.



Архитектура проекта:



Сравнение разворачивания инфраструктуры с подходом IaC и HandOps

Используя Terraform

- + Инфраструктуру можно быстро развернуть одинаково в разных окружениях.
- + Есть контроль изменений. конфигурация хранится в Git, есть plan, review и история изменений.
- Сложность настройки нужно знать понятие state, providers, dependency management и т.д.
- Drift — нужно управлять состоянием и следить, чтобы ручные изменения не расходились с Terraform.

Используя ручную настройку

- + Простота настройки.
- + Гибкость – точечные изменения по факту.
- Высокий риск человеческой ошибки.
- Сложность масштабирования

Процесс отработки восстановления

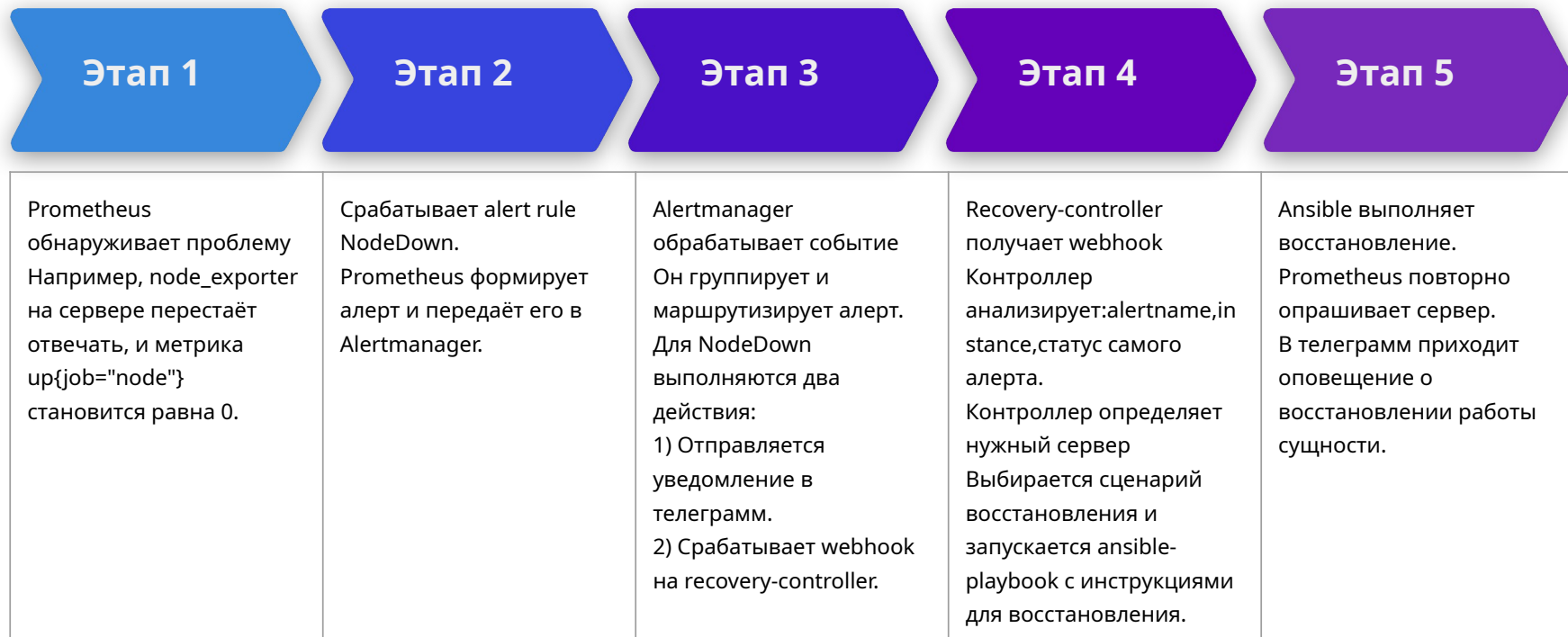
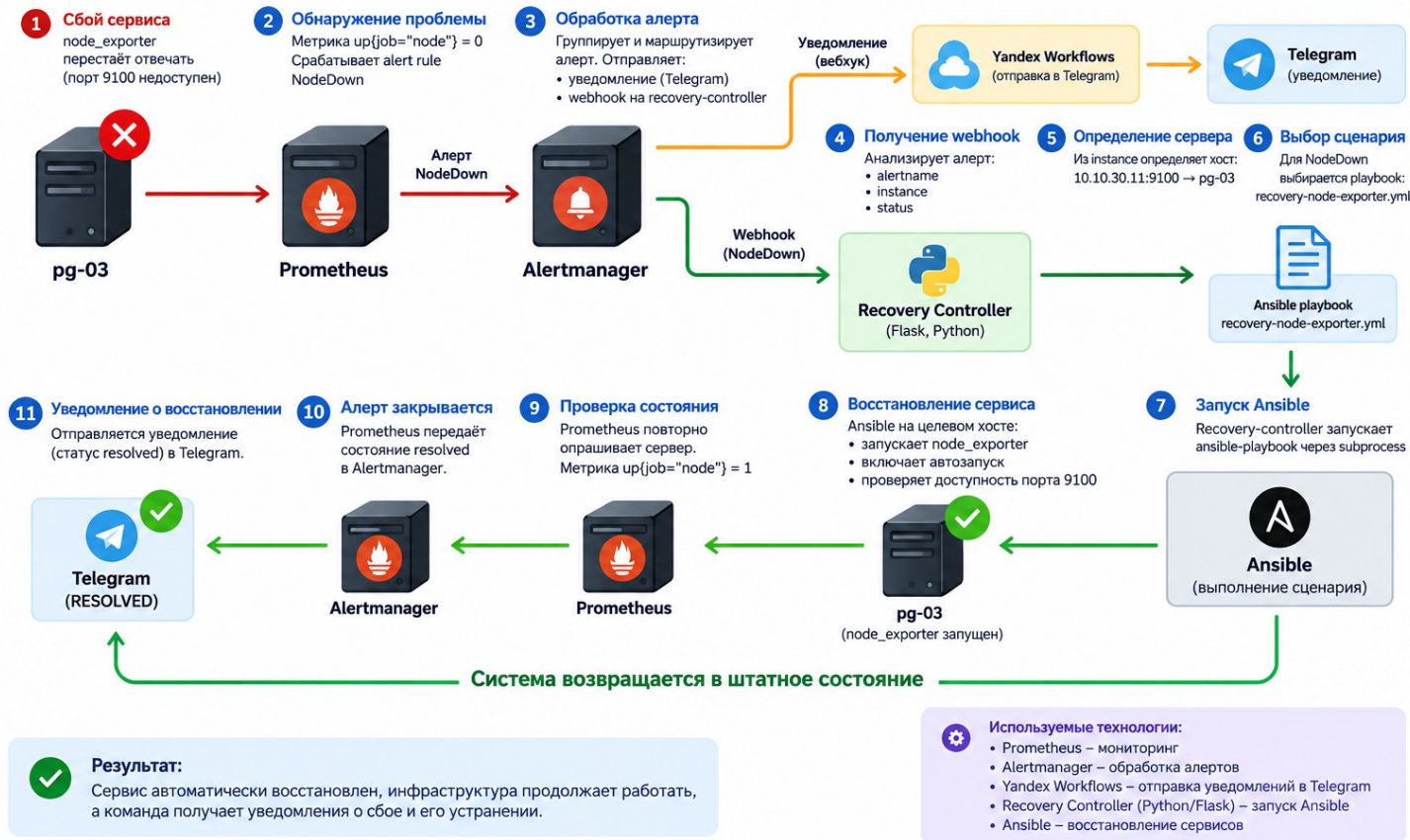


Схема отработки восстановления:

Схема отработки автоматического восстановления (с использованием recovery-controller)



Сервис авто-recovery

```
odd@ansible-01:~$ sudo journalctl -u recovery-controller -f
Sep 14 09:30:14 ansible-01 systemd[1]: Started recovery-controller.service - PostgreSQL HA Recovery Controller.
Sep 14 09:30:15 ansible-01 python3[749]: * Serving Flask app 'controller'
Sep 14 09:30:15 ansible-01 python3[749]: * Debug mode: off
Sep 14 09:30:16 ansible-01 python3[749]: 2026-09-14 09:30:15,723 INFO WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
Sep 14 09:30:16 ansible-01 python3[749]: * Running on all addresses (0.0.0.0)
Sep 14 09:30:16 ansible-01 python3[749]: * Running on http://127.0.0.1:5000
Sep 14 09:30:16 ansible-01 python3[749]: * Running on http://10.10.40.10:5000
Sep 14 09:30:16 ansible-01 python3[749]: 2026-09-14 09:30:15,723 INFO Press CTRL+C to quit
^C
odd@ansible-01:~$ sudo systemctl status recovery-controller.service
● recovery-controller.service - PostgreSQL HA Recovery Controller
   Loaded: loaded (/etc/systemd/system/recovery-controller.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-14 09:30:14 UTC; 21min ago
     Main PID: 749 (python3)
        Tasks: 1 (limit: 2313)
       Memory: 22.1M (peak: 22.4M)
          CPU: 373ms
     CGroup: /system.slice/recovery-controller.service
             └─749 /usr/bin/python3 /home/odd/postgresql-ha/ansible/recovery/controller.py

Sep 14 09:30:14 ansible-01 systemd[1]: Started recovery-controller.service - PostgreSQL HA Recovery Controller.
Sep 14 09:30:15 ansible-01 python3[749]: * Serving Flask app 'controller'
Sep 14 09:30:15 ansible-01 python3[749]: * Debug mode: off
Sep 14 09:30:16 ansible-01 python3[749]: 2026-09-14 09:30:15,723 INFO WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
Sep 14 09:30:16 ansible-01 python3[749]: * Running on all addresses (0.0.0.0)
Sep 14 09:30:16 ansible-01 python3[749]: * Running on http://127.0.0.1:5000
Sep 14 09:30:16 ansible-01 python3[749]: * Running on http://10.10.40.10:5000
Sep 14 09:30:16 ansible-01 python3[749]: 2026-09-14 09:30:15,723 INFO Press CTRL+C to quit
odd@ansible-01:~$ cat /etc/systemd/system/recovery-controller.service
[Unit]
Description=PostgreSQL HA Recovery Controller
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=odd
WorkingDirectory=/home/odd/postgresql-ha/ansible
ExecStart=/usr/bin/python3 /home/odd/postgresql-ha/ansible/recovery/controller.py
Restart=always
RestartSec=5

[Install]
```

Пример вызова ansible сценария для восстановления:

```
def run_playbook(playbook, recovery_host):  
    playbook_path = f"{ANSIBLE_DIR}/playbooks/{playbook}"  
  
    command = [  
        "/usr/bin/ansible-playbook",  
        "-i",  
        INVENTORY,  
        playbook_path,  
        "-e",  
        f"recovery_host={recovery_host}",  
    ]  
  
    logger.info(  
        "Running recovery: host=%s playbook=%s",  
        recovery_host,  
        playbook  
    )  
  
    try:  
        result = subprocess.run(  
            command,  
            cwd=ANSIBLE_DIR,  
            capture_output=True,  
            text=True,  
            timeout=180  
        )  
  
        logger.info(  
            "Recovery finished: host=%s playbook=%s rc=%s",  
            recovery_host,  
            playbook,  
            result.returncode  
        )
```

Описание виртуальной машины в Terraform

```
resource "yandex_compute_instance" "vm" {
  for_each = var.vms

  name      = each.key
  hostname  = each.key
  zone      = each.value.subnet == "bastion" ? "ru-central1-a" : var.subnets[each.value.subnet].zone
  platform_id = "standard-v3"

  allow_stopping_for_update = true

  resources {
    cores      = each.value.cores
    memory     = each.value.memory
    core_fraction = each.value.core_fraction
  }

  boot_disk {
    initialize_params {
      image_id = data.yandex_compute_image.ubuntu.id
      type     = each.value.disk_type
      size     = each.value.disk_size
    }
  }
}
```

Выводы

1. Terraform провайдер для Yandex Cloud можно установить на территории РФ и РБ только через зеркала.
2. 😜 Важно! Зафиксировать образ используемый для создания виртуальных машин, для того, чтобы избежать пересоздания машины при применении новых изменений на инфраструктуру.
3. На текущем этапе Patroni кластер PostgreSQL является наилучшим решением с наиболее высоким уровнем доступности и открытым кодом.
4. Часть конфигурационных файлов лучше хранить в гите для автоматизации развертывания инфраструктуры.
5. ❗ Ошибки совершенные на тестовом стенде – помогают избежать проблем на проде.

Вопросы и рекомендации



если есть
вопросы



если вопросов нет



**Ошибки — неизбежная
часть любой активной
деятельности, а полный
отказ от действий — это и
есть главная ошибка**

**Спасибо за
внимание!**



OTUS

